CrossAudit: A Git-Native, Cross-Vendor Audit Loop for Agentic Science

Zhaohe Dong
University of Cambridge
zd314@cam.ac.uk

Yuhao Chen
University of Wisconsin–Madison
ychen2546@wisc.edu

July 2026

Abstract

Autonomous "AI scientist" systems increasingly generate, execute, and review research. In the frontier systems we survey, the reviewing agent is an internal critic chosen by the same operator (often from the same model family or vendor as the generating agent) exposing supervision to self-preference bias and to blind spots shared across a common training pipeline; the supervision trace, moreover, usually lives in opaque platform logs rather than in an artefact a third party can replay. We present **CrossAudit**, a lightweight, infrastructure-minimal protocol for supervising autonomous research pipelines, built on three commitments. First, heterogeneity: every experiment increment produced by a generator agent is audited by an agent from a different model vendor, against a versioned, human-authored rulebook (the Constitution). Second, a git-native ledger: experiments, audit reports, verdicts, disputes and escalations are all commits, so the supervision history can be re-read, diffed, and cited by anyone. Third, graded human oversight: deterministic, model-free checks and hard inconsistencies gate the pipeline, judgement calls do not, and a bounded revision loop escalates unresolved blockers to a human principal.

We specify the protocol as eight invariants; describe a public reference implementation (GitHub Actions plus a few hundred lines of dependency-light Python) that *targets* them and implements a subset (deterministic checks, reply validation, receipts, anchored callbacks; receipt-verified fail-closed admission is specified but not yet enforced); and report a live deployment of a closely related variant in a computational-chemistry pipeline whose generator (Claude-based), auditor (GPT-lineage), and compute (cloud HPC) are mutually decoupled.

In an exploratory seeded-defect trial (30 synthetic increments, 43 defects; its intended blinding voided by a cross-vendor audit of our own repository, whose findings we adopt and commit alongside the data), a same-family auditor scored 38/43 at both scoring tiers with zero false-positive blockers, while a cross-vendor auditor scored 41/43 at the lenient tier but 37/43 at the strict tier and returned BLOCKED on all thirty increments; because recall under the frozen scorer rises with finding volume (the arms emitted 17, 63, and 113 findings), permutation-corrected agreement places the two model arms within noise at the lenient tier and reverses their order at the strict tier. The trial shows that vendors disagree, not that either is better; the strongest evidence in the repository is the committed record of cross-vendor audits of this paper itself. We argue that cross-vendor audit trails of this kind are a practical basis for accountable machine science, and one a single researcher can adopt today.

1 Introduction

Two things motivated this paper. The first is the arrival of genuinely agentic research systems: pipelines in which large language model (LLM) agents propose hypotheses, write and execute code, run experiments, and draft manuscripts with limited human intervention. Sakana's AI Scientist produced machine-generated papers end-to-end [1, 2] and the line of work has since been published in *Nature* [3]; DeepMind's AI co-scientist organises hypothesis generation as a multi-agent tournament [4]; autonomous laboratories close the loop through to physical synthesis [7, 8]. Publishers and funders are already being urged to respond institutionally [11].

The second is older and more familiar: science's continuing struggle with reliability. In one large survey, most researchers had failed to reproduce a published result [10]. Agentic pipelines raise the stakes on both sides of this ledger. They can enforce discipline no human sustains (every increment logged, every parameter recorded), or they can mass-produce plausible, well-formatted, wrong results faster than any human audit culture can absorb.

So the question is not whether machine-generated science should be supervised, but by what mechanism, and by whom. Current frontier systems embed supervision *inside* the system: a reflection agent, an automated reviewer, a ranking tournament [1, 4]. These internal critics share a structural weakness: critic and creator are instantiated from the same model or, at best, the same vendor's training pipeline. LLM evaluators are known to favour their own generations (self-preference bias, with self-recognition causally implicated [5]), and judge-position and style biases compound the problem [6]. Two agents that share a training distribution plausibly share blind spots, and a reviewer sharing the author's blind spots risks approving the author's characteristic errors. Meanwhile, the supervision trace of most agentic systems (who flagged what, what was revised, what was waved through) lives in ephemeral context windows or proprietary platform logs, invisible to the communities that will be asked to trust the resulting science.

Contributions. We make four contributions.

1. We articulate the *same-source supervision problem* in agentic science: internal critique by same-vendor models is structurally exposed to correlated failure (§2).
2. We specify **CrossAudit**, a supervision protocol defined by eight invariants (vendor heterogeneity, ledger completeness, citation validity, determinism-first, bounded revision, graded interruption, receipt binding, fail-closed admission) that together specify a re-inspectable, third-party-auditable (up to self-declared vendor identity, §5) supervision history (§3).
3. We describe a reference implementation requiring no infrastructure beyond two git repositories, a CI service, and two model API keys, together with a live deployment of a closely related variant in the first author's computational-chemistry pipeline (§4).
4. We give an explicit threat model, including the attacks the protocol does *not* withstand, and situate the design against plausible alternatives (§5–§6).

CrossAudit is deliberately modest in its claims. It does not certify scientific truth; no textual audit can. It certifies something narrower and checkable: that every increment of a machine-generated research record passed, or failed, a named set of versioned rules, applied by a configured second-vendor agent with no write access to the increment's repository (identity self-declared; §5), with the committed exchange preserved in the ledger, public wherever the operator publishes it (parsed reports today; raw model request/response capture is revision-2 work). We contend that this narrower property is the right first target for accountable machine science, plausibly for the same reason continuous integration (not formal verification) became the load-bearing quality mechanism of modern software.

2 Related work and the same-source problem

Agentic science systems. The AI Scientist [1, 2, 3] automates the ideation–experiment–writeup–review cycle, including an internal automated reviewer scored against human reviewer data. The AI co-scientist [4] structures generation–reflection–ranking–evolution agents into a self-improving tournament over hypotheses, with reported expert-validated outcomes in drug repurposing and antimicrobial resistance. Language agents for literature synthesis reach or exceed expert performance on retrieval-grounded tasks [9]. In the laboratory, Coscientist executed chemistry protocols from natural-language goals [7] and A-Lab ran autonomous materials synthesis campaigns [8]. Across all of these, quality control is architecturally *internal*: reviewer agents, reflection stages, or tournament judges chosen and hosted by the same operator, inside the same platform, with no commitment to vendor heterogeneity, and frequently instantiated from the same model family as the generators they judge.

The pattern persists through the 2025 generation of research agents: Curie hardens agentic experimentation with internal rigour modules [12], and Agent Laboratory attaches critic agents to its own pipeline [13]. Commercial platforms now advertise the transparency half of our proposal: Kosmos promises reports in which "every conclusion can be traced" to code or literature via its platform [14]. Traceability of that kind is audit by the generating operator on its own infrastructure, valuable, but with no described commitment to a second-party auditor or to an off-platform, replayable ledger; this paper is about that distinction.

Why internal critique is not enough. Panickssery et al. demonstrate that LLM evaluators recognise their own generations and rate them preferentially, with fine-tuning experiments providing evidence that self-recognition capability causes the inflation [5]. Zheng et al. catalogue further judge pathologies, position, verbosity, and self-enhancement biases, even in strong models [6]. These results concern single-model self-evaluation; we hypothesise the mechanism generalises: models sharing a training pipeline are likely to share stylistic priors and knowledge gaps, putting a same-vendor critic at the wrong end of the bias distribution just when it matters. We call this the *same-source supervision problem*. The remedy of drawing judges from different model families has precedent: Verga et al. score generations with a panel of evaluators from three model families precisely to dilute intra-model bias [15], and the same intuition has begun to appear in developer tooling as cross-vendor code review.[1] CrossAudit's claim is accordingly not the heterogeneity idea itself, but its packaging as a supervision protocol for research: versioned rules, a replayable ledger, deterministic precedence, and bounded escalation, run against a live pipeline. Cross-vendor pairing does not eliminate correlated error, frontier models plausibly train on largely overlapping public corpora, so where the shared literature is wrong, both can be confidently wrong together, but it forecloses the demonstrated configuration of the best-documented bias in this setting (self-preference) and is intended to decorrelate vendor-idiosyncratic failure modes. What cross-vendor pairing cannot fix, a model-free layer must (§3, **I4**).

Supervision as an artefact. Software engineering solved an analogous trust problem not by making programmers infallible but by making quality mechanisms *inspectable*: version control, continuous integration, review trails. Reproducibility initiatives in science point the same way, the unit of trust is the auditable record, not the author's competence [10]. Deterministic screening of the research record has its own lineage: statcheck recomputes reported statistics across the psychology literature [16], and the Problematic Paper Screener runs mechanical detectors over millions of published papers [17], post-publication analogues of our deterministic check layer, which moves checks of that character upstream, before a claim ever leaves the repository. At the ledger end, blockchain-based research-integrity infrastructures pursue tamper-evidence with far heavier machinery;[2] CrossAudit's position is that git (ubiquitous, free, and already attached to CI, permissions, and issue tracking) is ledger enough for the supervision trail. CrossAudit transplants the inspectability stance to agentic science: the supervision history itself becomes a first-class, versioned, citable research artefact. Table 1 compresses the comparison.

3 The CrossAudit protocol

3.1 Objects and roles

CrossAudit supervises a pipeline in which a **Generator** agent (vendor A) produces discrete *experiment increments*, commits under a designated path of a **Science Repository**. Each increment is audited by an **Auditor** agent (vendor $B \neq A$) against a versioned, human-authored rulebook, the **Constitution**, held with

[1] E.g. MindStudio, "Cross-Vendor AI Agent Review: Why Claude Should Review Codex's Code and Vice Versa" (June 2026), `https://www.mindstudio.ai/blog/cross-vendor-ai-agent-review-claude-codex`, as accessed 30 July 2026, a workflow recommendation, without versioned rules, ledger, deterministic precedence, or escalation semantics.

[2] E.g. the bloxberg consortium, `https://bloxberg.org`.

Table 1: Supervision properties of related systems, as published. • = held by design; ∼ = partial or informal; – = absent. PoLL is an evaluation technique rather than a supervision loop; statcheck and the Problematic Paper Screener are deterministic by construction but run post-publication, where CrossAudit moves checks of that character before a claim leaves the repository. The CrossAudit column describes the protocol specification; §4 states which invariants the public reference implementation enforces versus targets.

	Critic from a different vendor	Versioned, decidable rules	Replayable public ledger	Deterministic precedence	Bounded loop, escalation
The AI Scientist [1]	–	–	∼	–	∼
AI co-scientist [4]	–	–	–	–	∼
PoLL judge panels [15]	•	–	–	–	–
Cross-vendor review advice (§2)	•	–	∼	–	–
statcheck, PPS [16, 17]	–	∼	∼	•	–
CrossAudit (this paper)	•	•	•	•	•

all *audit reports* in a separate **Audit Repository**. A **Deterministic Check Layer** (DCL), plain scripts with no learned component: schema validation, unit and dimension analysis, convergence assertions, provenance completeness, runs before any model reads anything. A **Human Principal** authors the Constitution and receives escalations, but sits outside the per-increment loop. Compute (in the reference deployment, jobs on cloud HPC) is decoupled from supervision: verdicts gate the *next* increment, never running jobs. Figure 1 traces one increment through the loop.

The two-repository split is separation of powers made mechanical: the Auditor holds no write access to the Science Repository, the Generator none to the Audit Repository, so neither agent holds credentials to alter the other's record; guarding each record against its *own* author additionally requires branch protection.

3.2 Eight invariants

An implementation is CrossAudit when it preserves the following. (**I7** and **I8** are revision-1 invariants, adopted from the cross-vendor audit of this repository described in §4.3.)

I1 — Heterogeneity. The Auditor's model family differs from the Generator's. This prevents the auditor from ever judging its own generations, the configuration in which self-preference bias has been demonstrated [5]; similarity-driven preference is thereby reduced, not eliminated. It is likewise designed to decorrelate vendor-idiosyncratic failure modes, an effect this paper does not measure (§4). Heterogeneity is, moreover, one of three isolations the protocol stacks: *parametric* (different weights), *contextual* (no shared memory, the Auditor never sees the Generator's prompts, reasoning, or conversational state, only committed artefacts, closing the anchoring and sycophancy channels through which shared context sways a judge [18]), and *permissive* (no shared credentials; §3.1).

I2 (Ledger completeness. Every protocol artefact) increment, report, verdict, dispute, escalation, and every change to the Constitution, is a commit (or an issue) in one of the two repositories. No supervision state lives only in model context or platform logs. The history is replayable by a third party from the repositories alone, replay of the *record*; the model calls behind it are stochastic and not re-executable, and the reference ledger currently preserves parsed reports rather than raw exchanges.

I3 — Citation validity. An audit report must cite the rule IDs it applied and the commit hash of the Constitution version in force. A report that cites nothing, or an unverifiable version, is *invalid* and treated as an Auditor failure, it escalates; it can never pass an increment.

I4 — Determinism first. The DCL runs before any LLM audit. A DCL hard failure yields `BLOCKED` regardless of any model's opinion, and no model may waive a DCL verdict. Rationale: heterogeneous models still train on largely overlapping public corpora and hence can share their errors; the

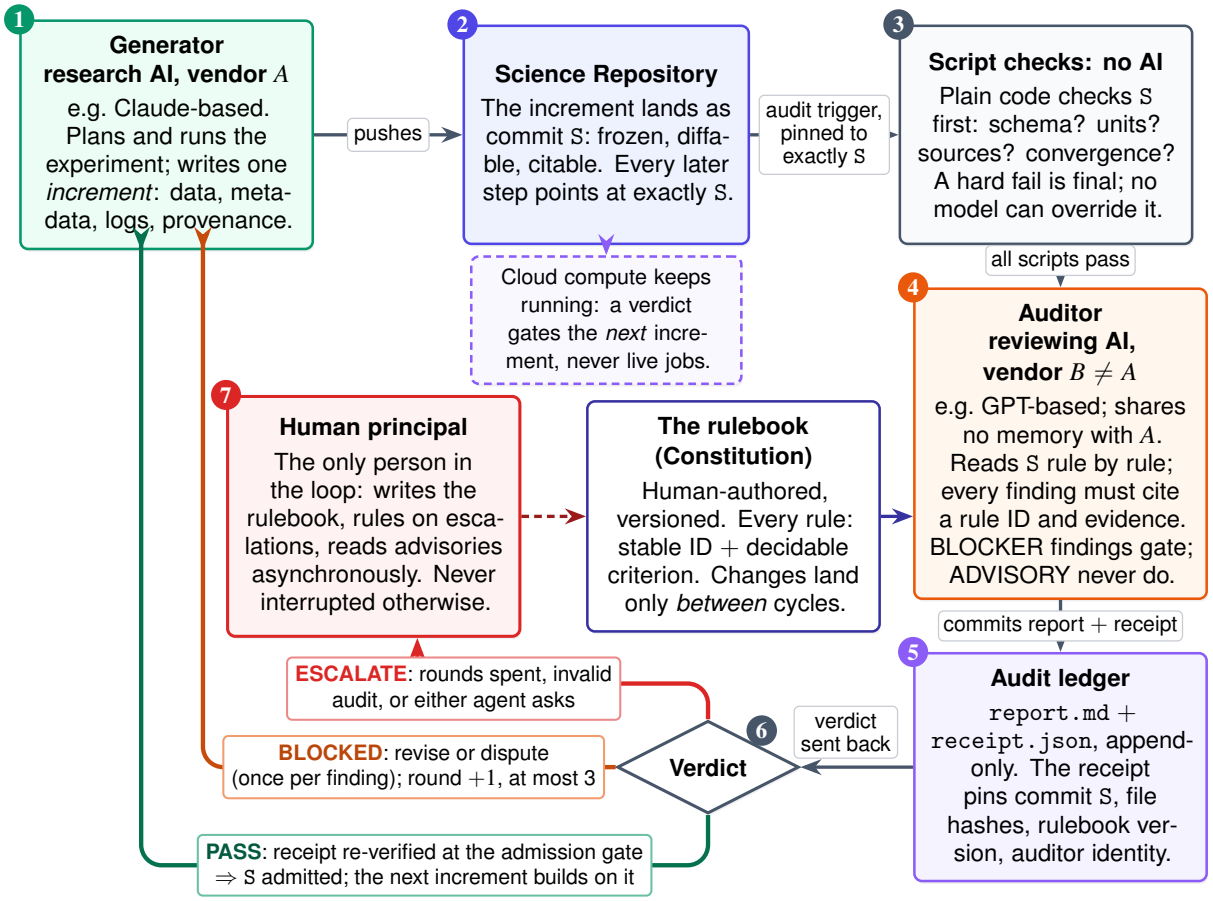

Figure 1: One experiment increment through CrossAudit, read clockwise from the top left. The Generator writes an increment, which is frozen as a commit; scripts with no learned component check it first, and their word is final in both directions (**I4**); an auditor from a different vendor (**I1**) then reviews the same pinned commit against the human-authored, version-pinned rulebook, citing a rule for every finding (**I3**), and commits its report and receipt to the audit ledger (**I7**). The two repositories are mutually unwritable: the Auditor cannot touch the science record, the Generator cannot touch the rules or the reports. Three fates follow: admitted, only on a verified receipt (**I8**); sent back, at most three rounds with each finding disputable once (**I5**); or escalated to the human, who writes the rules and is interrupted only by these cases (**I6**); rule changes take effect between cycles, never mid-round, so the Generator never faces a moving target. Compute is decoupled: a verdict gates the next increment, never running jobs, and silent stalls are caught by a periodic dead-letter sweep. Every artefact is a commit or an issue (**I2**), so a third party can replay the whole history.

supervision layer whose failure modes are least entangled with any model is one with no learned component at all.

I5 — Bounded revision. At most `max_rounds` Generator–Auditor exchanges per increment (default three). Exhaustion escalates. This is the oscillation guard: without it, two mutually-correcting agents can disagree indefinitely, each round locally plausible and the pair jointly divergent.

I6 — Graded interruption. Findings carry exactly two severities. `BLOCKER` (objective defect: DCL failure, internal contradiction, missing provenance, method–declaration mismatch) gates the increment. `ADVISORY` (judgement: parameter taste, style, scope) is recorded for asynchronous human reading and never gates. Humans are interrupted only by `ESCALATE`. Running compute is never interrupted at all.

I7 — Receipt binding. Every verdict is a *receipt* bound to the audited commit SHA; to an artefact manifest (paths and content hashes) that the Audit Repository derives from that SHA itself, never from caller-supplied payload; to the Constitution, check-layer, and prompt versions applied; to the auditor's declared identity; and to the ledger commit of the report. A verdict that binds less is not a receipt.

I8 — Fail-closed admission. Protected actions (admitting the next increment, production submission, claim publication) proceed only on verification of a valid, current, matching receipt. Absent, stale, conflicting, unbound, or model-audit-less receipts deny by default and escalate: no audit, no admission.

Two severities only; in our judgement every additional level invites severity-inflation negotiation between the agents; where a finer distinction is needed it belongs in the rule's acceptance criterion, not in the escalation lattice.

3.3 The Constitution

The Constitution is the protocol's quality ceiling: the Auditor's blocking power may not, by rule, exceed the rules it is given (miscitation is caught by the dispute channel, not prevented), and the Generator is free to exploit whatever the rules fail to say. CrossAudit therefore treats it as law rather than documentation. Every rule carries a stable identifier (`CA-DATA-001`), a severity, a path scope, a *decidable* acceptance criterion, and a rationale. Decidability is the load-bearing requirement: a criterion a competent reader can evaluate against named artefacts without further judgement. What cannot be phrased decidably is either demoted to `ADVISORY` or (better) mechanised and moved into the DCL. Meta-rules govern the audit itself: missing evidence is a finding, never an implicit pass (`CA-META-001`); a report without citations is invalid (`CA-META-002`, enforcing **I3**); the Auditor may not issue a `BLOCKER` without citing a `BLOCKER`-severity rule, it does not legislate taste (`CA-META-003`); and matters beyond the increment-plus-Constitution evidence base must escalate rather than be guessed (`CA-META-004`). Constitution changes are commits reviewed by the human principal; the loop can propose rules but never enact them. (The name deliberately echoes, but must not be confused with, Constitutional AI [19]: there, a constitution steers a model's training-time self-critique; here, it is a runtime rulebook applied by a structurally independent auditor and versioned in the ledger.)

3.4 Rounds, disputes, termination

Round k for an increment: the Auditor publishes report R_k with a verdict. On `BLOCKED`, the Generator either *revises* (a new commit, opening round $k{+}1$) or *disputes* specific findings by rule ID, with grounds. A disputed finding returns to the Auditor once: it is withdrawn with a logged reason, or upheld. An upheld dispute at the round bound escalates. Disputes exist because the Auditor is fallible in both directions, a false-positive `BLOCKER`, silently obeyed, degrades the science as surely as a false negative, and the one-shot rule prevents the dispute channel from becoming the oscillation it exists to control. Termination is structural: each increment ends in `PASS` or in human hands after at most `max_rounds` exchanges.

4 Reference implementation and deployment

4.1 Mechanics

The reference implementation[3] uses no infrastructure beyond GitHub itself and two model API endpoints. A push under `experiments/` triggers a cross-repository dispatch carrying the commit SHA; the audit side derives the audited scope from the SHA itself and the round from its own report index, caller-supplied payload is treated as untrusted hint. The Audit Repository's workflow checks out the science repository *pinned at the audited SHA*, asserts **I1** from configuration, runs the DCL, then invokes the Auditor via an OpenAI-compatible endpoint with the Constitution, the DCL output, and the increment content as fenced data. The report is committed to the ledger; the verdict is dispatched back; the science repository routes it, proceed, revise, or open an escalation issue. The whole implementation is three workflow files and two Python programs (a check runner and an auditor runner) of a few hundred dependency-light lines; configuration is a single `crossaudit.yml` declaring vendors, `max_rounds`, and escalation policy.

Status against the invariants. Following two rounds of cross-vendor audit of this repository (§4.3; both reports and dispositions committed under `audits/`), the reference implementation moves *toward* revision 1 without yet reaching it. Implemented: audited scope derived from `git diff` at the pinned SHA; rounds derived from the ledger's report index rather than generator-controlled trailers; per-cycle `receipt.json` binding resolved SHA, per-file manifest, Constitution hash, auditor identity, and input-bounds status; truncated, unreadable, or symlinked inputs escalate rather than pass; the reply validator rejects fabricated rule citations, coverage-free reports, and PASS-with-blocker replies; keyless runs yield `DCL_ONLY`, which the router treats as non-admission; and the verdict router fetches the receipt from the *configured* audit repository at the exact full-SHA report commit and matches its bound fields before acting. A third cross-vendor read (also committed) then found the sharper failure mode: the controller state machine and full receipt verifier existed but were not wired into the execution path, dry-run admission, a hard-coded report path that broke on disputed rounds, a constitution check that could never fire. Those wiring defects are fixed; what remains unimplemented is the deployer-side binding of the admission status to branch protection, and raw model-exchange capture in the audit runner. The public implementation should therefore be read as *targeting* **I1**–**I8** and implementing selected checks; the remaining distance is enumerated in `ROADMAP-R2.md`.

Three details with protocol weight. First, the auditor runner validates the model's reply against a fixed schema (verdict, findings with severities and rule citations) and any malformed or citation-free reply is converted to `ESCALATE` (**I3**); a familiar failure of LLM judges (the confident empty report) is thereby converted from silent passage into an escalation (when the DCL has also hard-failed, the verdict remains `BLOCKED` and the auditor failure is recorded as an `audit_integrity` field in the receipt; router-side escalation on that field is revision-2 work). Second, increment content is passed to the Auditor as fenced, size-bounded content with the system prompt instructing that embedded instructions be ignored, a best-effort defence whose residual risk §5 states plainly; the reference Auditor has no tools. Third, verdict synthesis is code, not model output: DCL hard failures dominate any model verdict (**I4**).

4.2 A live deployment

A closely related variant of CrossAudit runs live in the first author's computational-chemistry pipeline (perovskite additive screening). The generation side is a Claude-based research agent that plans calculations, launches them on Alibaba Cloud E-HPC and a GPU worker, and pushes every increment to a private science repository whose audit policy, an epistemic-status ledger separating established, bounded, and provisional claims, together with a standing retraction log, is itself a committed, versioned document. A controller opens each cycle against a fixed science commit. The audit request pins a SHA-256 manifest of

[3] `https://github.com/dongzhaohe321418-lab/crossaudit` (MIT licence).

every tracked file. A Codex-CLI auditor (OpenAI lineage, so **I1** holds against the Claude-based generator) writes four artefacts into an append-only per-cycle directory in the audit repository: a human-readable report, a machine-readable result, run metadata, and a per-file hash manifest. All sit on one long-lived `audit` branch that later cycles may not rewrite. The controller enforces this by validating every audit commit before recording a cycle as final, which detects a rewrite rather than preventing one; prevention would need branch protection. The controller's own orchestration state deliberately lives outside both repositories, so tampering by either agent with the record of its own supervision is detectable against controller-held state. Three deliberate divergences from the reference design should be named plainly: the deployed auditor is a tool-bearing CLI agent rather than the reference's toolless one, contained instead by that commit validation and the append-only cycle layout; the externalised controller state means the deployment's history is fully replayable from the repositories *plus* that state, a weaker property than **I2**'s repositories-alone ideal; and findings carry a four-level severity ladder feeding scope-level gates, rather than the protocol's two levels, a live experiment running against §3.2's two-level argument, whose outcome the telemetry will eventually adjudicate. Every input and output lives on GitHub; the system is designed so that any single component (generator, auditor, controller host, workstation) can fail without loss of ledger state.

Two refinements of the deployed system go beyond the reference implementation and are worth adopting upstream. First, findings are *evidence-typed*: each carries a policy citation (section and rule ID), a deterministic check identifier where applicable, and a copy-paste *reproduce* command, so a human can re-derive any deterministic finding in seconds. Second, blocking is *scope-granular*: a finding blocks named scopes, production-job submission, claim publication, rather than the pipeline wholesale, refining **I6**'s graded-interruption principle. Human authority enters through escalation, plus one deployment-specific addition: a standing principal-approval gate on production launches, recorded in the ledger. The loop is live in the plainest sense: at the time of writing, its most recent cycle (opened and closed within six minutes on the morning of writing) returned a `BLOCK` decision on deterministic grounds (manifest-hash disagreements and commit-provenance defects), freezing production submission and claim publication until resolved. Audits gate the record and future submissions, never running jobs: E-HPC work in flight is never interrupted. A redacted public mirror of the deployment's finalised cycles is maintained in the reference repository (`ledger-mirror/`; scientific magnitudes masked, structure, rule citations and reproduce-command shapes verbatim). The ledger is young (at the time of writing it holds its first finalised cycle) so operational telemetry (cycle counts, verdict mix, escalation and dispute rates, per-cycle cost) is deferred to a revision once volume has accumulated.

Costs for the reference design are modest. The DCL is free. A single fenced-increment audit call scales with the size of the increment, not of the computation behind it, estimated at cents for a few hundred lines of results and metadata, while the experiment it certifies may represent hours of HPC time. (The deployed variant's tool-bearing auditor reads more and costs more; per-cycle cost reporting is deferred with the rest of the telemetry.) Auditing costs roughly what it costs to read the documentation, not what the compute behind it cost. Two caveats: multi-round exchanges multiply audit calls, and escalations spend human time, which is the truly scarce resource here.

The deployment's role is existence proof for the deployed variant: the loop has run unattended, at single-researcher scale, on commodity infrastructure. The one concrete catch reported above is attributable to the deterministic layer; the deployment record claims no LLM-layer catches.

4.3 An exploratory seeded-defect trial, audited and corrected

We attempted a pre-registered, sealed seeded-defect pilot. A cross-vendor audit of this repository, a second-vendor agent reviewing the generating agent's work, its BLOCK-grade report and our dispositions committed at `audits/`, subsequently voided the "sealed" framing and several companion claims. We adopt its findings in full, and report the corrected, exploratory results, which prove more instructive than the intended experiment.

Design and execution. A deterministic generator produced 30 synthetic increments in the reference format (10 clean, 20 defective; 43 seeded defects over ten registered classes, four mechanisable, six requiring reading), with a SHA-256 of the defect key committed at registration. Three arms ran over identical inputs with the identical Constitution: the deterministic check layer; a same-family arm (fresh-context Claude subagents, one per increment, no shared memory; session sampling, a deviation we disclose); and a cross-vendor arm (`gpt-5.1`, temperature 0, run by the frozen runner in CI).

What the audit found. We checked each finding and accepted all of them. The key file was public from the registration commit onward, our mistake, a mis-pathed ignore rule and is in any case derivable from the public seeded generator, no blinding claim survives, only procedural blinding of fresh-context auditors, which the ledger cannot verify. The headline same-family score depended on a scoring-map expansion made after outputs were committed: the frozen map yields 38/43; the adjudicated map, with corrections logged, 43/43, the former is the reportable score, the latter exploratory. The deterministic layer's five L6 catches contradict the registered taxonomy (the class was misregistered as model-only; its flag component is mechanisable). The trial is component-level, corpus read directly, no dispatch pipeline, no DCL-precedence, not an end-to-end run of the reference implementation. And the trial's own outputs violate **I3**: the runner recorded no Constitution hash, a defect one of the auditors itself flagged (below).

Arm	Recall lenient / strict	Chance floor lenient / strict	Blockers on 10 clean	Verdicts (of 30)
Deterministic layer	17/43 [a]	4.8 / 4.2	0	25
Claude-family (subagents)	38 / 38 [b]	22.4 / 16.3	0	30
GPT-family (`gpt-5.1`, frozen runner)	41 / 37 [c]	31.8 / 24.4	26	20

[a] 12/12 of its registered scope + 5/5 cross-hits on misregistered L6. [b] 43/43 under post-hoc adjudicated scoring. [c] all ten clean increments blocked (10/10 false-block verdicts); many findings letter-valid on inspection; per-finding adjudication pending. Strict-tier scores assemble rule and location evidence across findings rather than binding one finding per defect, and so overstate defect-level localisation. Chance floors: 2000-shuffle permutation null, seed 20260731; `experiment/results/NULLCHECK.json`.

The vendor split is the result. The cross-vendor arm scored 41/43 lenient but 37/43 strict, emitted 113 findings to the same-family arm's 63, and returned BLOCKED on *all thirty* increments, a constant verdict, which a stub auditor that blocks unconditionally would reproduce exactly (including the 20/30 verdict accuracy). A permutation test (2000 shuffles of the increment-to-defect map, which holds each arm's outputs fixed and so preserves its output volume and citation habits; seed 20260731, `experiment/score_nullcheck.py`) puts the random floors at 4.8, 22.4, and 31.8 of 43 for the three arms, so the headline lenient comparison is within noise once corrected, and the strict-tier order reverses (chance-corrected agreement 0.81 same-family vs 0.68 cross-vendor). Inspection of the clean-increment blockers still suggests many are letter-valid readings of an ambiguous rule, but per-finding adjudication is not logged, and as framed the claim was unfalsifiable: any disagreement would have counted as support. `CA-DATA-001` requires "each numeric entry" in the results file to carry unit and source; the corpus's convergence blocks carry units but no source. The same-family auditor (like the check scripts, and like the corpus author) read the rule by its intent and raised nothing; the foreign auditor enforced its letter. Several further clean-increment blockers flag the harness's failure to supply the Constitution hash: the same **I3** violation the human-relayed audit independently found. What survives is narrower but real: on identical inputs the two vendors read the same rulebook differently, which is **I1**'s premise, but the trial measures disagreement, not discrimination, and a 10/10 false-block rate on clean increments means that in this configuration every increment escalates, making §6's O(escalations) economics conditional on a far better-calibrated auditor or a sharper Constitution. The practical outcome is a Constitution amendment (scope the rule decidably).

What we can still claim. This is a component-level, checklist-aligned trial (the prompt names the defect families the corpus injects) on template-generated data, with the same-family arm's execution

self-attested and n=43; zero blockers on ten clean increments bounds that arm's false-positive rate only below ≈26% (one-sided 95%), and its advisory burden on clean increments was 5 findings across 5 of 10. None of this is confirmatory evidence about detection rates. What the trial does establish: the model layers read (26–31 catches beyond the scripts' scope); vendor heterogeneity produced observably decorrelated readings; and the repository's own disciplines, append-only outputs, logged adjudication, a committed cross-vendor BLOCK with itemised dispositions, were applied to the evaluation itself.

5 Threat model and limitations

Below is what CrossAudit withstands and what it does not.

- **Self-preference and vendor-correlated error** are addressed by **I1**. *Corpus*-correlated error is not: where the shared literature is wrong or outdated, both vendors can agree confidently and wrongly. Mitigation is **I4** for everything mechanisable, and the human's asynchronous reading of `ADVISORY` trails for what is not. This residual cannot be assumed reducible by adding more LLM auditors, since all plausibly train on largely overlapping public corpora.
- **Prompt injection.** An increment could embed text addressing the Auditor. The Auditor runs toolless against a fixed reply schema with content fenced as data; and because DCL verdicts are non-overridable, injection cannot waive the layer where hard defects are caught. Injection shaping the *LLM-side* report remains partially open, as it does for LLM-judge systems generally, and this bullet describes the reference implementation; the deployed variant's tool-bearing auditor forgoes the toolless defence, a divergence §4 names together with its containment limits.
- **Auditor failure**, empty, malformed, or rule-free reports, converts to escalation (**I3**), never to silent passage. The cost is human attention, which is the correct currency for an unreliable auditor.
- **Oscillation and deadlock** are bounded by **I5**; the failure mode degrades to a human decision with the full exchange on the ledger, which is the intended floor.
- **Goodharting the Constitution.** A capable Generator will satisfy the letter of decidable rules. Decidability makes this visible rather than solved: the human reviews rule–outcome drift on the ledger and legislates accordingly. The Constitution is a living document; its growth from escalations is the system learning.
- **Collusion** has no out-of-band channel in the reference design, coordination must ride the committed artefacts, where it is preserved for inspection; encoded coordination *within* artefacts, and any channels a tool-bearing auditor acquires, remain possible. Convergent style over many rounds is the realistic soft risk, met by Constitution review cadence.
- **The operator.** CrossAudit supervises the agents, not the principal. Selective publication, pre-publication history rewriting, and audit re-rolling by the operator sit outside the protocol's guarantees; external anchoring (signed pushes, transparency logs) would extend tamper-evidence to the operator, and until then "third-party-auditable" should be read as *internally consistent as published*. Relatedly, the DCL scripts and the Auditor's system prompt must live under the same ledgered, human-reviewed change control as the Constitution, a Generator-editable check layer would hollow out **I4**.
- **Vendor identity is self-declared.** The ledger proves an exchange occurred, not the identity of the models behind OpenAI-compatible endpoints; **I1** is asserted from configuration, not verified cryptographically. Attested inference or vendor-signed responses would close this; today it is an honest limit on what "third-party-auditable" can mean. (The reference example moreover shares one dispatch token across both workflows, so credential separation is likewise configuration, not proof.)

- **Confidentiality.** The protocol ships every increment (possibly unpublished, patentable material) to a second vendor's API. Data governance of that call (agreements, retention, disclosure risk) sits outside the protocol and must be settled per deployment.
- **Scope.** CrossAudit audits the research *record*, not the world: it cannot detect a miscalibrated instrument, a wrong dataset honestly described, or fraud upstream of the repository. It is also currently specified for single-generator pipelines; multi-agent generation raises attribution questions we do not address here.

6 Discussion

Continuous integration for scientific claims. The ambition is what CI achieved for software: not proof of correctness, just a cheap and inspectable floor under quality, with human judgement redeployed to where it discriminates (the rules, the escalations, and the science) rather than spent uniformly on every increment. The analogy holds, we believe, in one further respect: CI spread without institutional permission, a single developer could adopt it unilaterally, and its artefacts (badges, logs) became social proof. One disanalogy must be owned: CI's checks are deterministic while CrossAudit's distinctive layer is a stochastic judge; the DCL exists precisely to keep the gate's hard core deterministic. CrossAudit is designed for the same adoption path: one researcher, two repositories, an afternoon.

Relation to platform-scale systems. CrossAudit is complementary to, not competitive with, systems such as the AI co-scientist [4] or the AI Scientist [3]. Those systems generate; CrossAudit supervises whatever generates. Indeed the sharpest version of our proposal is addressed to exactly such platforms: internal critique stages appear architecturally straightforward to re-instantiate across vendor lines and to externalise onto public ledgers (the organisational and confidentiality costs are not ours to estimate), and the resulting audit trails would give publishers and funders, currently being asked to trust process descriptions [11], something they can actually inspect.

What the ledger buys science socially. A complete supervision history, public where the operator publishes it, changes the character of machine-generated results. A claim arrives not as prose plus reputation but as prose plus its audit record: which rules it was checked against, what was contested, what a differently-trained model conceded or refused. Reviewers can spot-check the ledger instead of re-deriving trust from nothing.

The ledger as a data asset. A CrossAudit deployment produces, as a by-product, something the field currently largely lacks: a corpus of *supervised machine science*. Each (increment, findings, dispute, resolution) tuple is a labelled example of scientific work judged by an independent party under explicit rules, high-grade training signal for better generator and auditor agents alike. Aggregated across deployments, failed audits form a public error taxonomy of agentic research: what machine-generated science actually gets wrong, per rule, per field, over time. Whoever runs the loop accumulates this asset; a community that runs it openly accumulates it as a commons.

Constitutions as community objects. Because the Constitution is a plain, forkable file with stable rule IDs, it can outgrow any single deployment: a research community can maintain a shared domain constitution the way communities maintain style guides and lint rules; a journal could publish the constitution it expects machine-assisted submissions to have been audited under, and accept the ledger link alongside the manuscript, exactly as data-availability statements are accepted today; funders could write "public supervision ledger" into terms. For regulated settings (pharmaceutical and clinical computation among them), the ledger's attributability and contemporaneous, hash-manifested record resemble properties audit regimes already demand of human work, though we claim compliance with no specific regime. And a repository badge (increments audited, escalations raised, constitution version) would do for machine

science what CI badges did for software: compress an inspectable process into a glanceable, verifiable signal. None of this requires new technical infrastructure; it requires only that the artefact exist, which a conforming deployment produces.

A ratchet for standards. The Constitution need not be static. Because rules are commits and receipts pin the version in force (**I7**), standards can tighten over time without ambiguity about which version governed which increment. The safe dynamics have three channels. First, shadow-mode promotion: a candidate stricter rule enters as `ADVISORY`, rehearses enforcement on every cycle without gating, and accumulates ledger evidence (hit rate, dispute rate, would-be blocking cost); the human promotes it to `BLOCKER` only when the record supports it. Second, threshold ratchets: numeric criteria tighten stepwise as the generator's competence grows, each step a commit with its rationale. Third, agent-proposed amendment: either agent may draft a rule change from what the ledger shows it, the loop can propose, but only the principal enacts. One rule matters most: standards freeze within a cycle and move only between cycles; an auditor that could raise the bar mid-round would face the generator with a moving target and dissolve the termination guarantee of **I5**.

From copied glue to an installable protocol. The reference implementation ships as copy-in glue: a deployer vendors `checks/` and `controller/` into an audit repository and edits three workflow files. Our planned next step is a versioned package (`pip install crossaudit`) exposing the same components behind a command-line surface: `crossaudit init` to scaffold the two-repository pair, `crossaudit audit` to run the DCL and the auditor against a pinned SHA, `crossaudit verify --admit` as the admission call in CI. The argument is not only convenience. Vendored copies drift, so two deployments can silently run different verifiers, and a receipt that pins the Constitution, the check sources, and the prompt, but not the machinery that verified and admitted on their basis, is bound one layer short of **I7**'s intent. A release-versioned verifier closes that layer: the receipt records the package version and distribution hash alongside the hashes it already carries, and the enforcement code becomes a pinned, citable dependency rather than an unversioned copy. Packaging also gives field adaptation a natural unit, check packs as extras (`crossaudit[compchem]`, `crossaudit[ml]`), so DCL coverage can circulate between groups the way we argue Constitutions should. The cost is a new trust surface, the package supply chain; hash-pinned installs and signed releases are the standard mitigations, and the verifier field in the receipt at least makes a drifted or substituted verifier visible after the fact.

A console for the human principal. The ledger is git-native on purpose, and stays so; but raw git is a demanding reading surface for the one human the protocol keeps in the loop, and **I6** makes that human's asynchronous reading load-bearing. The second planned artefact is therefore a small application over the ledger, a supervision console: per-increment cycle timelines (increment, report, verdict, rounds, receipt status), the advisory backlog sorted by rule and hit rate, an escalation inbox that assembles what an adjudication actually needs (the round history, the diffs, the generator's dispute and the auditor's grounds), and the telemetry the standards ratchet reads (which rules fire, which are disputed, what a promotion would have blocked). One design rule carries protocol weight: the console writes nothing of its own. Every action it offers, closing an escalation, withdrawing a finding, amending the Constitution, materialises as a commit or an issue through the same authenticated paths the agents use, so the ledger remains complete (**I2**) and the console remains derivable from it. A supervision interface with a private database would accumulate exactly the unversioned, unreplayable state the protocol exists to exclude.

Management by exception, at research scale. As agent labour becomes cheap, the binding constraint of agentic science shifts from compute to human attention. CrossAudit's economic content is a reduction in *gating* cost from $O(\text{increments})$ to $O(\text{escalations})$, conditional on escalation volume staying low. Our own trial shows the failure mode plainly: the cross-vendor arm's 10/10 false-block rate would make escalations $O(\text{increments})$, so the economics hold only with a calibrated auditor and a decidably scoped Constitution, and must be measured, not assumed. This is the managerial precondition for one researcher

credibly directing several agent pipelines at once, and it should in principle extend beyond science to agentic work products that arrive as discrete increments whose quality is partially rule-expressible, from code to quantitative analysis, a conjecture this paper does not test.

7 Conclusion

CrossAudit reframes supervision of autonomous research from a platform feature into a public, versioned artefact produced by structurally independent reviewers. Its ingredients are plain: a second vendor, a rulebook with stable IDs, scripts that run before any model, a bounded loop, and git underneath. We chose them because a single researcher can adopt all of this without asking anyone's permission. The protocol's eight invariants are a deliberately small set that jointly buys structurally independent review, a re-inspectable history, and human authority at the boundary; everything else in this paper is reference detail, and the reference repositories are public. An AI scientist should not grade its own homework. With two repositories and two API keys, it no longer has to.

Acknowledgements. The CrossAudit reference implementation and the deployment described in §4 were developed with the assistance of the very class of agentic tools this paper proposes to supervise.

References

[1] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024.

[2] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv:2504.08066*, 2025.

[3] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026.

[4] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, et al. Towards an AI co-scientist (later retitled "Accelerating scientific discovery with Co-Scientist"). *arXiv:2502.18864*, 2025.

[5] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.13076.

[6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.05685.

[7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[8] N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, et al. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624:86–91, 2023.

[9] M. D. Skarlinski, S. Cox, J. M. Laurent, J. D. Braza, et al. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv:2409.13740*, 2024.

[10] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[11] Editorial. AI scientists are changing research — institutions, funders and publishers must respond. *Nature*, 651:853–854, 2026.

[12] P. T. J. Kon, J. Liu, Q. Ding, et al. Curie: Toward rigorous and automated scientific experimentation with AI agents. *arXiv:2502.16069*, 2025.

[13] S. Schmidgall et al. Agent Laboratory: Using LLM agents as research assistants. *arXiv:2501.04227*, 2025.

[14] Edison Scientific. Kosmos: An AI scientist for autonomous discovery. Announcement, November 2025. https://edisonscientific.com/news/announcing-kosmos.

[15] P. Verga, S. Hofstätter, S. Althammer, et al. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv:2404.18796*, 2024.

[16] M. B. Nuijten, C. H. J. Hartgerink, M. A. L. M. van Assen, S. Epskamp, and J. M. Wicherts. The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, 48:1205–1226, 2016.

[17] G. Cabanac, C. Labbé, and A. Magazinov. The 'Problematic Paper Screener' automatically selects suspect publications for post-publication (re)assessment. *arXiv:2210.04895*, 2022.

[18] M. Sharma, M. Tong, T. Korbak, et al. Towards understanding sycophancy in language models. *arXiv:2310.13548*, 2023.

[19] Y. Bai, S. Kadavath, S. Kundu, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.